

五、架构反思：流水线 vs Agent 循环——修复一个 Bug，一定需要 Agent 吗？

核心问题： Agentless 的固定三阶段流水线在哪些场景下胜过 Agent 动态循环？
什么时候循环不可替代？

OODA 定位： 质疑循环本身 / L→R→V 三阶段全覆盖

从上一节的问题切入

上一节结尾留了一个问题：

如果固定流水线在定位阶段就能击败 Agent 循环，那整个 L→R→V 流程都用流水线行不行？修复一个 Bug，一定需要完整的 Agent 循环吗？

上一节讲了 Observe——三种 Localization 策略的对比，Agentless 的三层漏斗以 32% 的解决率和 \$0.70 的成本击败了 SWE-agent (12.47%/\$2.53) 和 AutoCodeRover (19%/\$0.45)。但那只是 Localization 阶段。

自然的追问：**如果 Localization 不需要循环，Repair 和 Validation 也不需要吗？**

本讲的核心对比框架：

Agentless 流水线：Localization → Repair → Validation (三阶段，无循环)
Agent 循环： while(true) { Observe → Orient → Decide → Act } (动态，有反馈)

问题：哪个更好？
答案：取决于任务。

前四讲分别优化了 OODA 循环中的各个环节——Act (第二讲)、Orient (第三讲)、Observe (第四讲)。本讲跳出单个环节的视角，**质疑循环本身的必要性**。

5.1 Agentless 的完整流水线：补全 Repair 和 Validation

第四讲详细讲了 Agentless 的 Localization (阶段一：三层漏斗)，本节补全后两个阶段。

阶段二：多候选采样修复（Repair）

Agentless 在修复阶段不是只生成一个 patch。它用高 **temperature 采样** 生成多个候选 patch。这背后有一个关键洞察：

模型单次生成正确修复的概率 $p \approx 0.2$

如果只生成 1 个 patch：成功率 $\approx 20\%$

如果生成 10 个候选 patch：至少一个正确的概率 $= 1 - (1-0.2)^{10} \approx 89\%$

这是**推理时计算（inference-time compute）换准确率**的策略——与 best-of-N 采样、self-consistency 同源。核心思路：模型单次不够准，但多次独立采样中，总有一个踩对。

Agent 方案用多轮试错达到类似效果（跑测试→看报错→再改），本质都是在搜索空间中找正确解，只是搜索策略不同：

	Agentless	Agent
搜索策略	并行采样 N 个候选	串行试错，每轮一个
反馈利用	无（每个候选独立生成）	有（前一轮的错误信息影响下一轮）
成本特征	可预测（固定 N 次调用）	不可预测（可能 3 轮也可能 30 轮）

并行 vs 串行、无反馈 vs 有反馈——这是两种根本不同的搜索哲学。Agentless 赌的是“广度”（多个独立候选），Agent 赌的是“深度”（每轮利用反馈缩小搜索范围）。

阶段三：测试驱动验证（Patch Validation）

用测试套件对所有候选 patch 做筛选，选出**通过率最高的**作为最终输出。

这个阶段有一个关键前提：**必须有现成的测试套件**。如果 repo 没有相关测试，Validation 阶段等于无效——Agentless 只能盲选。Agent 方案可以动态写测试、用 lint 检查、甚至手动验证，灵活度高得多。

三阶段的成本拆解

Agentless 论文（Table 2-4）提供了详细的成本分解：

Localization（三层漏斗） \$0.15 ← 最便宜

Repair（多候选采样） \$0.29 ← 主要成本

Validation (测试筛选)	\$0.26
总计	\$0.70

三个阶段的成本相对均匀，没有哪个阶段特别昂贵。对比 SWE-agent 的 \$2.53/issue（其中 Localization 的多轮搜索可能占 60%+ 的 token 消耗），Agentless 全流程的成本只有 SWE-agent 的 28%。

5.2 search/replace 的殊途同归

在进入核心对比之前，值得注意一个重要的结构性发现：Agentless 和 Claude Code 在编辑格式上独立收敛到了相同的范式。

Agentless:	<<<< SEARCH / ==== / REPLACE >>>>	← 内容定位
Claude Code:	old_string → new_string	← 内容定位
Unified Diff:	@@ -10,5 +10,6 @@	← 行号定位

为什么两个独立的项目不约而同地选择了 search/replace？因为另一个选项——**unified diff（行号定位）**——对模型来说极易出错。

Diff 依赖**行号定位**，而模型对数字天然不敏感。行号偏移一行就 apply 失败，上下文行数不匹配也会失败。**内容定位**（search/replace）用代码原文本身做锚点，天然抗漂移——只要代码文本没变，无论前面插了多少行，匹配都不受影响。

真正的差异不在编辑格式，而在**校验策略**：

CC 源码实证（src/tools/FileEditTool/utils.ts）：CC 的 Edit 工具有三层校验：1. 精确匹配优先：直接搜索 old_string 2. 引号规范化 fallback：curly quotes ↔ straight quotes 统一后再匹配（小模型常混淆引号类型）3. 唯一性检查：old_string 匹配多处且 replace_all=false → 拒绝执行 4. staleness 检查：编辑前校验文件自上次 Read 后是否被外部修改

对比 Agentless 的“生成后直接 apply”——CC 选择了**宁可报错重试，也不悄悄改错地方**。

这个设计选择的背后是架构差异：CC 是 Agent 循环，报错后下一轮可以纠正；Agentless 是流水线，没有“下一轮”——所以它把容错放在了 Validation 阶段（用测试筛选），而不是 Repair 阶段。

5.3 核心对比：复杂度与能力的 trade-off

Agent 循环的工程复杂度

回顾前几讲中提到的 CC 工程细节，汇总一下 Agent 循环需要付出的“复杂度税”：

CC 的 queryLoop() 有：

- 12 个退出点 (return)
- 7 个继续点 (continue)
- 6 层上下文压缩机制 (第三讲 §3.4)
- Nudge/Recovery 机制 (第三讲 §3.4)
- 权限检查、Hook 系统
- 工具并发编排 (只读工具并发、写工具串行)

Agentless 的流水线有：

- 3 个阶段
- 每阶段 1 次 LLM 调用
- 无状态管理
- 无上下文压缩
- 无权限系统

CC 源码实证（`src/query.ts`，`queryLoop()` 行 241-1729）：1488 行的主循环函数。12 个 return 退出点分别处理：`blocking_limit`、`image_error`、`model_error`、`aborted_streaming`、`prompt_too_long` (×2)、`completed` (×2)、`stop_hook_prevented`、`aborted_tools`、`hook_stopped`、`max_turns`。7 个 continue 继续点处理：`context collapse` 重试、`reactive compact` 重试、`max_output_tokens` 升级/恢复、`stop hook` 阻断、`token budget` 续延、工具结果收集后的下一轮。

这 19 个分支路径（12 退出 + 7 继续）中的每一个，都是 Agent 循环必须处理的边缘情况——而 Agentless 一行都不需要。

这不是说 Agent 设计过度。这些复杂度是**真实的工程需求**——只要你允许模型在循环中自由交互，就必须处理所有可能的异常。流水线的简单性来自它**放弃了自由度**。

完整对比表

维度	Agentless（流水线）	Agent（CC）
编辑格式	search/replace（自创语法）	search/replace (old_string → new_string)

维度	Agentless（流水线）	Agent（CC）
唯一性校验	无，生成后直接 apply	有，old_string 不唯一则拒绝执行
搜索策略	并行采样 + 测试筛选	串行试错 + 反馈修正
容错机制	多候选中选一个能过测试的	严格校验 + 模型看到错误后多轮重试
上下文管理	无（每阶段独立调用）	6 层压缩纵深（第三讲 § 3.4）
循环复杂度	0（无循环）	12 退出 + 7 继续（1488 行）
SWE-bench Lite	32%	23%（SWE-agent + Claude 3.5 Sonnet）
成本/issue	\$0.70	\$1.62-\$2.53
优势	无状态，可并行，成本可控	能利用中间反馈，适应意外
劣势	不可回溯，定位错则全盘皆输	可能死循环，上下文膨胀

5.4 Agentless 的“脆性”——固定流水线的致命弱点

Agentless 的三阶段**严格串行、不可回溯**。每个阶段的输出是下一个阶段的输入，错误会级联放大：

Localization 定位错误（例如选错了文件）

↓

Repair 基于错误定位生成 patch → 10 个候选全部是错的

↓

Validation 在 10 个错误 patch 中选"最好的" → 还是错的

↓

总成本 = \$0.70，全部浪费

回顾第四讲引用的 AutoCodeRover 论文失败案例分析（Section 6.3）：**18% 的失败是因为定位到了错误的文件**——连正确的战场都没找到。对 Agentless 来说，这 18% 完全不可恢复。

Agent 在实际工作中可以动态调整方向：

Agent Round 4: Edit(file_A.py) → 修改代码

Agent Round 5: Bash("pytest") → 测试失败

Agent Round 6: [Thought] "file_A 不对，错误指向 file_B"

```
Agent Round 7: Read(file_B.py) → 重新定位
Agent Round 8: Edit(file_B.py) → 修改正确的文件
Agent Round 9: Bash("pytest") → 测试通过 ✓
```

这种**动态纠偏能力**——“试一下→发现不对→换个方向”——是 ReAct agent 的核心优势，也是 Agentless 流水线无法提供的。

用 Boyd 的框架理解：Agentless 做了一次 OODA 循环就停了（单轮决策）；Agent 可以持续循环直到收敛（多轮决策）。单轮决策效率高但脆弱，多轮决策冗余但鲁棒。

5.5 工程权衡结论：按任务难度匹配策略

不存在“流水线永远好”或“Agent 永远好”的答案。正确的选择是**按任务难度匹配策略**：

任务光谱：

简单 ←

→ 复杂

Pipeline - 单文件 bug fix - 明确的错误信息 - 有现成测试覆盖 - 变更范围可预见 成本：\$0.70 SWE-bench Lite ✓	Agent Loop - 跨文件重构 - 模糊的需求描述 - 需要动态探索 - 需要写新测试 - 涉及架构决策 成本：\$1-3+ SWE-bench Full ✓
--	---

成本的另一个维度：模型选择

CC 源码实证：CC 的模型定价揭示了 Agent 循环成本的精细结构：

模型	Input (per M)	Output (per M)	典型角色
Opus 4.6	\$5	\$25	规划、复杂推理
Sonnet 4.6	\$3	\$15	日常编码（大多数用户默认）
Haiku 4.5	\$1	\$5	轻量辅助（摘要、校验）

一个 10 轮 Agent 循环（每轮 ~2K input + ~1K output），用不同模型的成本差异： - Opus 4.6: ~\$0.35/session - Sonnet 4.6: ~\$0.21/session - Haiku 4.5: ~\$0.07/session

对比 Agentless 的 \$0.70/issue（含 Localization + Repair + Validation 全流程），**Agent 单轮的模型成本并不高，高的是轮数不可预测**——3 轮解决和 30 轮解决的成本差 10 倍。Pipeline 的优势不在于单次调用更便宜，而在于**调用次数是确定的**。

Agentless 用简单流水线达到了 Agent 方案约 70-80% 的效果，证明了大量问题不需要复杂的 Agent 循环。这个发现的工程意义很大：

- 对于 CI/CD 中的自动 bug fix：Pipeline 更适合（可预测成本、可并行、无状态）
- 对于开发者的交互式编码助手：Agent 更适合（需要动态响应、处理意外）
- 对于大规模代码维护：两者结合——Pipeline 处理 80% 的简单问题，Agent 处理 20% 的复杂问题

CC 自身也在朝着混合方向演进——Subagent 机制（下一讲）本质上是“Pipeline 思维在 Agent 框架内的体现”：主 Agent 规划拆解（类似流水线的 Localization），spawn 子 Agent 并行执行（类似 Agentless 的多候选并行采样），子 Agent 的结果汇总后决策（类似 Validation 的筛选）。

5.6 [Demo 5 • 图解] 同一个 Bug，两条修复路径

用同一个 Bug 修复任务，并排看两条技术路线的完整执行流程。

Agentless 流水线



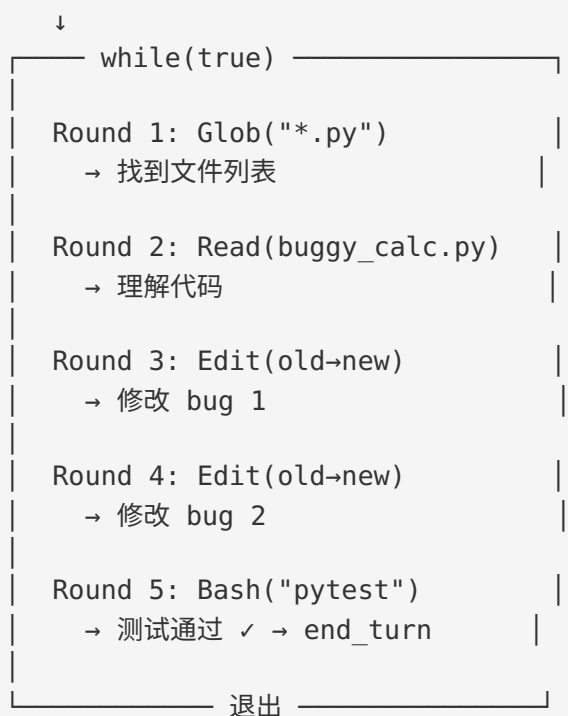


总计：~5 次 LLM 调用，无循环

成本：\$0.70

Agent 循环（以 Demo 3 的 buggy_calc.py 为例）

[Issue 描述]



总计：5 轮交互，有反馈循环

成本：\$1-3（取决于轮数）

对比标注：

维度	Agentless	Agent
信息流	单向（一次性决策）	双向（有反馈）
失败恢复	不可能（错了就白费）	Round 6 可以重来
搜索效率	并行 10 个候选	串行逐轮试错
额外开销	无	搜索阶段消耗 token
可预测性	高（固定成本）	低（可能 3 轮也可能 30 轮）

当 Agent 在 Round 5 测试失败、Round 6 分析错误、Round 7 重新修复时——这就是 Pipeline 做不到的。但如果 Agent 在 Round 3 就修对了，它多花了 2 轮搜索的 token 其实是“浪费”——Pipeline 的 Localization 用更少的 token 完成了同样的定位。

5.7 本节小结：循环是奢侈品，不是必需品

回到认知地图，本节的视角跳出了单个 OODA 环节——不是优化循环中的某个步骤，而是质疑循环本身：

	Localization	Repair	Validation
Observe	(第四讲)		
Orient	(第三讲)		
Act	(第二讲)		
★ 全局 (本节)	★ Agentless 三层漏斗	★ 多候选 采样	★ 测试筛选 验证
	← 流水线覆盖 L + R + V 全链路 →		

三个核心收获：

1. 流水线 ≈ Agent 的 70-80%

Agentless 用固定流水线在 SWE-bench Lite 上达到 32%，成本 \$0.70。同期 SWE-agent 12.47%/\$2.53、AutoCodeRover 19%/\$0.45。**大量问题不需要复杂的 Agent 循环**——这是对前四讲的重要校准。

2. 循环的核心价值 = 动态纠偏

Agent 循环不可替代的地方在于“试一下→发现不对→换个方向”。流水线的“脆性”在于定位错误不可回溯，全链路成本白费。循环提供了容错能力，但代价是工程复杂度（CC 的 1488 行 queryLoop、12 退出 + 7 继续）和不可预测的 token 成本。

3. search/replace 是独立收敛的共识

Agentless 和 CC 从完全不同的起点到达了相同的编辑范式——内容定位优于行号定位。但 CC 额外加了唯一性校验，因为 Agent 循环的架构允许“报错→下一轮纠正”。这说明编辑格式的设计与整体架构紧密耦合——不能脱离架构讨论单个组件的设计。

下一节的问题是：无论是流水线还是单 Agent 循环，都有一个共同限制——当任务复杂到一个上下文窗口装不下时怎么办？一个 Agent 搞不定的大任务，如何拆分给多个 Agent 协作？